INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO RIO GRANDE DO SUL *CAMPUS* BENTO GONÇALVES

TÉCNICO EM INFORMÁTICA PARA INTERNET

VICENZO DE SOUZA CIOATO

RELATÓRIO DE ESTÁGIO NO CTA

BENTO GONÇALVES

2026

VICENZO DE SOUZA CIOATO

RELATÓRIO DE ESTÁGIO NO CTA

Relatório escrito para satisfazer ao requisito obrigatório para a graduação no curso de técnico em informática para internet pelo instituto federal de educação, ciência e tecnologia do Rio Grande do Sul campus Bento Gonçalves

Orientador: Prof. Thiago Sávio Carbone

BENTO GONÇALVES

2026

RESUMO

O principal objetivo deste trabalho acadêmico é demonstrar as atividades realizadas durante o período de estágio, por meio do qual todo o conhecimento adquirido durante os três anos do curso técnico em informática para internet foi aplicado. O estágio em questão tem, como principal meta, capacitar os alunos para o mercado de trabalho com uma vivência de 200 (duzentas) horas de carga horária em ambientes profissionais, proporcionando uma experiência que é também diferencial para que se possa permanecer com uma carreira de sucesso na área da computação. O documento apresenta um detalhamento profundo sobre a experiência no período de estágio, abordando as atividades realizadas, tecnologias utilizadas, aprendizados e entre outros, abordando tanto o *front-end* quanto o *back-end* . As referidas atividades foram realizadas no Centro Tecnológico de Acessibilidade do IFRS, e são frutos de registros realizados periodicamente, garantindo a veracidade das informações e permitindo a conclusão de que o objetivo do estágio foi cumprido com êxito.

Palavras-chave: estágio; *front-end*; *back-end*; acessibilidade; *chatbot*.

LISTA DE FIGURAS

FIGURA 1 - Comunicação com o bot..... 15
FIGURA 2 - Geração vetorial..... 16
FIGURA 3 - Envio dos vetores..... 16
FIGURA 4 - Tela de login..... 19
FIGURA 5 - Esquematização do banco de dados..... 20
FIGURA 6 - Barra lateral..... 21
FIGURA 7 - Visão geral da interface..... 22
FIGURA 8 - Tela de login definitiva..... 26
FIGURA 9 - Aba de conversa..... 26

LISTA DE ABREVIAÇÕES

API - Application Programming Interface

CTA - Centro Tecnológico de Acessibilidade

CSS- Cascading Style Sheets

GPT - Generative Pre-trained Transformer

HTML - HyperText Markup Language

IFRS - Instituto Federal do Rio Grande Do Sul

ID - Identificador

IA - Inteligência artificial

JS - Javascript

LLM - Large language model

LGPD - Lei geral de proteção de dados

PAV - Projeto de acessibilidade virtual

PDF - Portable Document Format

SDK - Software Development Kit

SECADI - Secretaria de Educação Continuada, Alfabetização, Diversidade e Inclusão

TI - Tecnologia da informação

VS code - Visual Studio Code

SUMÁRIO

1 INTRODUÇÃO..... 7
2 SOBRE O LOCAL..... 8
3 REVISÃO BIBLIOGRÁFICA..... 9
3.1 CONCEITOS..... 9
3.2 DIRETRIZES DE ACESSIBILIDADE..... 10
3.3 FERRAMENTAS..... 10
3.4 LINGUAGENS..... 10
4 ATIVIDADES REALIZADAS..... 12
4.1 ESTUDO SOBRE A IA..... 12
4.2 PLANEJAMENTO DO DESENVOLVIMENTO..... 13
4.3 A PRIMEIRA VERSÃO..... 14
4.3.1 Comunicação com a IA..... 14
4.3.2 Alimentando a IA..... 15
4.3.3 Conclusão da primeira fase..... 17
4.4 A SEGUNDA VERSÃO..... 17
4.4.1 Sistema de login..... 17
4.4.2 Banco de dados..... 19
4.4.3 Barra lateral..... 20
4.4.4 Conversas e mensagens..... 21
4.4.5 Protótipo da interface..... 22
4.5 TERCEIRA VERSÃO..... 23
4.5.1 Uso do MongoDB..... 23
4.5.2 Modelo de IA local..... 24
4.5.3 Refatoração de código..... 24
4.5.4 Recursos de usabilidade e interface..... 25
4.5.5 Documentação com agente de IA..... 25
5 CONCLUSÃO..... 27
5.1 DIFICULDADES DO PROCESSO..... 27
5.2 PERSPECTIVA DO PROJETO..... 27
5.3 CONSIDERAÇÕES FINAIS..... 28
REFERÊNCIAS..... 29

1 INTRODUÇÃO

O Curso Técnico em Informática para Internet, ofertado pelo Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul – *Campus* Bento Gonçalves possui como elemento obrigatório da formação do aluno a conclusão de um período de 200 horas de estágio relacionado à tecnologia. Esse processo contribui diretamente para a capacitação dos futuros profissionais alunos do curso, além de permitir a vivência prática de tudo o que é ensinado durante os anos de ensino médio, fora aprendizados próprios adquiridos no período de estágio.

O interesse de realizar o período no CTA surgiu após a realização de um trabalho integrador durante o segundo ano do curso, em que foi proposto o desenvolvimento de um sistema para o NAPNE, e eventualmente houve o contato com o CTA e seus projetos. A primeira comunicação ocorreu por e-mail, e seguiu neste meio até uma visita presencial em que foi definido o prazo do estágio e o que seria desenvolvido. Durante essas conversas, foram alinhadas as atividades do estágio, seu período e horários, e entre outros detalhes.

As atividades passaram por todas as etapas do desenvolvimento de um software, desde seu planejamento inicial até a implementação de ideias avançadas e presentes no mercado real de trabalho, sempre sob os conselhos do supervisor de estágio Lael Nervis e do parceiro de trabalho Jason Piloti, além do professor orientador Thiago Carbone, que sempre se colocou à disposição para ajudar com qualquer eventual dúvida.

Desta maneira, esse relatório tem como objetivo relatar, completamente e de forma metódica, o processo de desenvolvimento e a experiência adquirida durante o período de estágio no Centro Tecnológico de Acessibilidade, que durou dos meses de abril até agosto, na área do desenvolvimento web.

Para a organização, tanto do processo de desenvolvimento do sistema quanto do relatório, consequentemente, o projeto foi separado em três principais versões, que acrescentavam consigo as principais funcionalidades e mecânicas que seriam requisitadas no sistema final, além de aprimorar as que já estavam presentes quando isso se demonstrasse como necessário.

2 SOBRE O LOCAL

O Centro Tecnológico de Acessibilidade(CTA), ligado à reitoria do IFRS e localizado no *campus* de Bento Gonçalves do instituto federal do Rio Grande do Sul, nasceu com o propósito de promover a acessibilidade pelos meios digitais, seja com recursos assistivos, testes ou adaptações em sites e editais, ou com tecnologias assistivas, construídas no próprio CTA. O local possui um laboratório de impressoras 3D e outras ferramentas semelhantes para a adaptação de recursos quando necessário.

Esse projeto passou por diversas mudanças e etapas desde a sua criação, ainda no ano de 2006 sob o nome de “projeto de acessibilidade virtual”. Nos anos posteriores, entre convênios e integrações com outras instituições, o PAV foi, no ano de 2013, oficialmente institucionalizado pela reitoria de extensão do IFRS, e posteriormente teve seu nome alterado para Centro Tecnológico de Acessibilidade.

Durante seus 20 anos de história, o CTA estabeleceu diversas parcerias para projetos com outras instituições. Dentre eles estão: o projeto de avaliação de acessibilidade *web*, em parceria com a *Symplicity Corporation*, que classifica a adequação de uma página para os parâmetros de acessibilidade, além de uma parceria com o Ministério da Educação, através da Secretaria de Educação Continuada, Alfabetização, Diversidade e Inclusão (SECADI).

A equipe do CTA é composta por sete pessoas, sendo, respectivamente, três servidores e quatro colaboradores: Bruna Poletto, a coordenadora do CTA; Lael Nervis, técnico em TI; Lucas Schwochow, Analista de Tecnologia da Informação; Andréa Poletto Sonza, assessora de ações afirmativas, inclusivas e diversidade; Bruno Kenji Nishitani Egami, docente do *campus* Farroupilha; Tiago Belmonte Nascimento, técnico do laboratório de física e eletrônica e Anderson Dall Agnol, assistente em administração.

3 REVISÃO BIBLIOGRÁFICA

Durante o período de estágio, houve o uso de diversos conceitos técnicos aprofundados que, consequentemente, serão mencionados no presente relatório. É justamente para o esclarecimento básico desses conceitos que se dá o conteúdo deste capítulo.

3.1 CONCEITOS

A inteligência artificial é um campo da ciência da computação cujo cunho técnico é desenvolver sistemas que realizam atividades às quais naturalmente requisitariam da inteligência humana, a partir da análise de altos volumes de informação para identificar padrões e gerar respostas sem ser programado especificamente para cada caso. Essa área é uma das mais importantes da tecnologia, pois está presente no dia a dia e nas mais diversas formas, como no corretor ortográfico do telefone celular, no algoritmo das redes sociais, e até mesmo em jogos eletrônicos.

No ano de 2022, o termo *chatbot* se popularizou após o lançamento da primeira versão pública do *chat GPT*, que consistia em um bate-papo inteiramente respondido por um modelo de inteligência artificial capaz de gerar respostas e realizar longas tarefas em questão de segundos. A tecnologia fez tanto sucesso que outras empresas criaram seus próprios modelos e iniciaram uma verdadeira corrida para ter o melhor modelo de IA.

A acessibilidade digital se resume pela construção de um *site* em que, independentemente de quem acessar, terá a mesma possibilidade de usufruir da experiência completa, eliminando barreiras que impediriam pessoas com alguma limitação de poder utilizar o sistema construído.

Uma API é como uma ponte ou tradutor entre diferentes sistemas de software, permitindo que eles se comuniquem e troquem dados. Em vez de recriar funcionalidades do zero, um desenvolvedor usa a API de outro serviço para acessar seus recursos. Ela define regras, pedidos e respostas padronizadas para que dois programas entendam um ao outro.

3.2 DIRETRIZES DE ACESSIBILIDADE

Para garantir uma experiência agradável aos usuários que necessitem de recursos de acessibilidade, o governo do Brasil possui um guia de boas práticas de acessibilidade guiado pela Web Content Accessibility Guidelines (WCAG), publicado pela empresa americana W3C. O guia apresenta quatro princípios de acessibilidade(Perceptível, Operável, Compreensível e Robusto) e três níveis de conformidade(A, AA e AAA) para avaliar se a página está adaptada às camadas mais básicas de acessibilidade.

3.3 FERRAMENTAS

Para a codificação do projeto, foi utilizado Visual Studio Code, um programa gratuito disponibilizado pela Microsoft e que é considerado o editor de código mais popular entre os programadores, por se propôr a ser customizável para a maioria das linguagens de programação.

Já para garantir um dos pilares do sistema, que é um funcionamento completamente local da IA, sem a dependência de qualquer sistema externo, foi utilizado o Ollama, um *software* que disponibiliza o download e a execução de diversos modelos de IA completamente de graça e de forma simplificada.

Composer é um gerenciador de pacotes e dependências para o PHP, permitindo que o usuário instale bibliotecas de funções prontas para uso com facilidade. Diferentemente de plataformas como o *sudo* ou o APT, o *Composer* funciona separadamente por projeto, semelhante ao *node*.

3.4 LINGUAGENS

O desenvolvimento do sistema criado durante o estágio demandou o uso de linguagens e frameworks das duas principais áreas: o *backend* e o *frontend*.

O sistema usa para sua estruturação a *HyperText Markup Language*, o HTML. Essa se trata de uma linguagem de marcação,e não de programação, e é a maneira como todas as páginas da web são estruturadas. Ele utiliza palavras-chave prontas fixas chamadas de *tags*, que sinalizam ao navegador que elementos renderizar na

tela. Apesar dos elementos serem fixos, as *tags* possuem atributos editáveis que permitem a edição da maneira como esse componente é renderizado.

Para a edição visual desses componentes é utilizado o *Cascading Style Sheets* (CSS), que permite o desenvolvedor modificar totalmente a aparência de qualquer elemento da página, seja individualmente, pelo sistema de *ID*, ou pelo sistema de classes, que modifica coletivamente elementos com a mesma marcação.

Já no âmbito do comportamento da página, para as interações dinâmicas com a interface, notificações de ações na tela e envio das requisições foi usado o *Javascript*, pela sua capacidade de adicionar dinamicidade aos elementos HTML.

O *backend* foi feito utilizando Laravel, um *framework* do PHP que agiliza a criação de diversos elementos do sistema que, se feitos com PHP puro, tomariam diversas horas do desenvolvimento e puderam ser feitos com um simples comando.

Para alguns elementos da interface foi utilizado o *Livewire*, uma biblioteca que complementa o Laravel e permite a interação direta entre o Laravel, o HTML e o *Javascript*, facilitando o desenvolvimento de partes da interface que interagem diretamente com o banco de dados.

Node.js é um ambiente de execução baseado no motor V8 do *Chrome* que permite que o *Javascript* seja executado fora do navegador, o utilizando como *backend* e permitindo gerenciar requisições de forma assíncrona e sem bloquear o funcionamento do código.

O banco de dados é uma tecnologia de armazenamento e consulta de dados, e neste caso foi inicialmente construído utilizando o banco de dados relacional MySQL. Porém, após algumas conversas sobre o andamento do projeto, foi sugerida a utilização de MongoDB, que além de ser mais apropriado para esse tipo de projeto por não exigir uma estrutura fixa para todos os elementos, possui uma velocidade de busca menor, melhorando o desempenho do sistema.

4 ATIVIDADES REALIZADAS

O estágio se iniciou no dia 17 de Abril, estendendo-se até 17 de Agosto. Porém, antes do início desse período, já havia sido decidido o projeto que seria desenvolvido. Em conversas com o supervisor de estágio Lael, foi mencionada a dificuldade para obtenção de informações sobre acessibilidade digital para desenvolvedores de aplicações *web*, de forma que estas só podiam ser encontradas em seus respectivos documentos, dificultando o processo de estudo e localização das informações importantes. Com isso, foi proposta a criação de um *chatbot* próprio do CTA, que funcionasse de forma inteiramente local para que assim pudesse ser mantido inteiramente nos servidores dedicados da reitoria do IFRS, sem custos externos. O sistema deveria ser alimentado com os documentos essenciais para a inclusão digital, como a ABNT da acessibilidade e as diretrizes de acessibilidade para conteúdo web (WCAG) 2.2. A ideia já havia surgido desde a popularização das IAs, mas até o momento havia sido considerada, pelos antigos estagiários do CTA, inviável de construir devido às restrições especificadas. Por conta disso, seria necessária uma série de pesquisas e estudos sobre os recursos disponíveis para avaliar a possibilidade de iniciar o desenvolvimento e pôr em prática o projeto.

Decidido qual seria o problema a ser solucionado, foi solicitado então um estudo breve sobre o processo de criação do *chatbot* para definir as tecnologias que seriam utilizadas no desenvolvimento, suas possíveis limitações e áreas que poderiam ser aprimoradas no futuro.

4.1 ESTUDO SOBRE A IA

Primeiramente, foi necessário realizar buscas sobre maneiras de manter uma IA funcionando localmente, ou ao menos de poder fazer perguntas diretas e exibir a resposta. Com isso, foi encontrado o *software* Ollama, que disponibiliza o *download* de diversos modelos de IA para executar pelo computador. O problema para esta opção era o processo de desenvolvimento: rodar todo o processo de pensamento lógico de uma IA localmente exige muito do computador, e trabalhar dependendo de respostas que tomam um minuto completo para terminar sua geração poderia se tornar um processo extremamente ineficaz enquanto feito em máquinas de menor

capacidade. Como alternativa, surgiu a API da *Groq*, que funcionava com um dos modelos do Ollama, mas sem a demora para as respostas. Com isso, era possível escrever um código que funcionaria exatamente igual quando migrado para o modelo de IA local, mas sem a demanda de tempo ou um computador extremamente avançado.

Tendo em mãos uma maneira de perguntar e receber uma resposta, o único recurso que ainda era necessário buscar eram as maneiras de fornecer as informações técnicas específicas para uma IA. Após algumas pesquisas, se optou pelo uso do banco de dados vetorial *Pinecone*. Diferente de outros bancos de dados, ele realiza suas buscas convertendo os dados em vetores numéricos e após isso faz uma aproximação semântica, ao invés da similaridade de palavras, sendo uma das principais ferramentas específicas para o desenvolvimento de *chatbots*.

Com o conhecimento destes processos da IA, já era possível concluir que a base do projeto seria plenamente possível, bastando apenas delimitar outras necessidades que deveriam se fazer presentes antes de iniciar o projeto, assim como o planejamento de como o processo de desenvolvimento deveria acontecer na prática.

4.2 PLANEJAMENTO DO DESENVOLVIMENTO

Com a confirmação de que o projeto seria possível, foi seguro partir para a análise de outros modelos de IA famosos, como o Claude, Gemini, Grok, e estabelecer *features* indispensáveis observáveis neles e que deveriam ser adaptadas em algum momento. Após essa análise, foi estabelecida uma organização baseada em três versões principais do projeto e o que deveria se fazer presente em cada uma delas. A primeira delas deveria apresentar um funcionamento básico da conversa, a segunda delas teria um sistema de contas inteiramente funcional, enquanto a terceira entraria com uma interface, funcionalidades para melhorar a experiência do usuário, IA local e entre outros. Durante o processo, o cronograma teve mudanças pequenas e não foi seguido à risca, mas serviu para guiar e auxiliar na organização como um bom norteador.

Para fundamentar o desenvolvimento, foram escolhidas para o desenvolvimento do *frontend* o HTML, o CSS e o *Javascript*, as três mais populares para esse âmbito do site. Já para o *backend*, se optou pelo PHP, que é a linguagem

trabalhada durante os dois últimos anos do curso e portanto foi considerada a mais adequada para o projeto. No banco de dados, se optou neste primeiro momento pelo *MySQL*, também por ser objeto de estudo nos anos de curso. A escolha inicial pelo Node.js ocorreu de forma experimental, motivada pelo seu ecossistema de SDKs, os quais proporcionaram maior agilidade ao ciclo de desenvolvimento.

Com o planejamento pronto e sem nenhum detalhe de planejamento deixado de lado, já era seguro iniciar o desenvolvimento do projeto, juntamente com o período de estágio.

4.3 A PRIMEIRA VERSÃO

O objetivo no começo desse nesse momento era, simplesmente, atender aos requisitos básicos de funcionalidade dos quais o sistema necessitava, ao mesmo tempo em que permitiu diversos testes e reorganizações do sistema, buscando possíveis problemas futuros que poderiam acontecer pela abordagem escolhida.

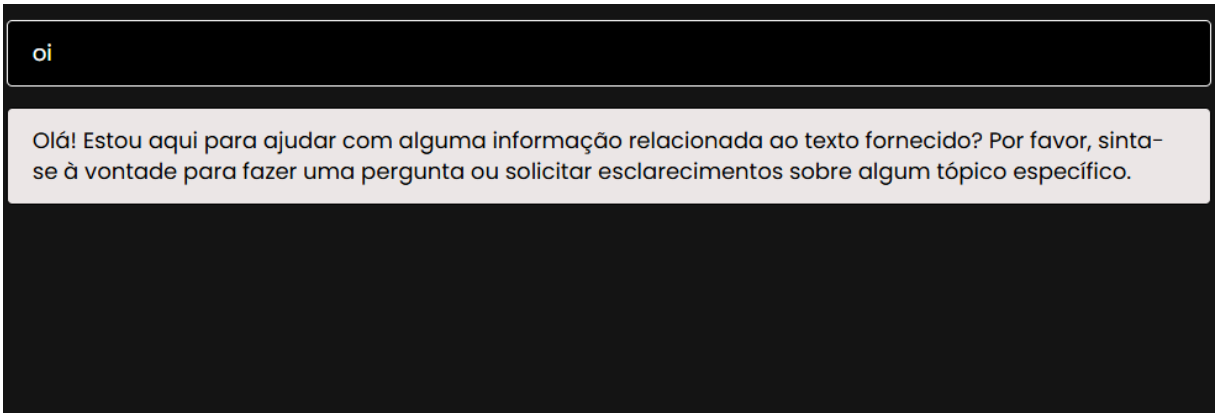
4.3.1 Comunicação com a IA

O ponto inicial do desenvolvimento do sistema foi, sem dúvidas, a comunicação básica com a API da *groq*, visto que seria com base nela que as alterações futuras se refletiriam.

Após pesquisas sobre o funcionamento da API, concluiu-se que a maneira mais simples para a construção do código fosse utilizando o pacote SDK oficial da API do *bot*, visto que o ambiente *Node.js* já seria necessário futuramente para a alimentação do banco vetorial. Mesmo não havendo um conhecimento prévio sobre o funcionamento do *Node.js*, ainda se optou pelo uso, visto que o projeto só utilizaria de recursos básicos do ambiente e não haveria a necessidade de um estudo avançado, além de ser um momento de testar abordagens.

O processo de desenvolvimento desta etapa foi simples, utilizando apenas conceitos básicos do *Javascript* aplicados no ambiente do *Node.js*, como requisições assíncronas e arquivos de *backend*.

FIGURA 1 - Comunicação com o bot



Fonte: Desenvolvida pelo autor (2026).

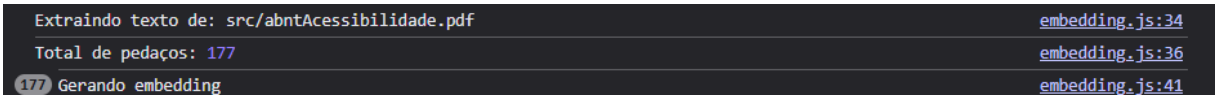
4.3.2 Alimentando a IA

Agora com a base em mãos, se iniciou o desenvolvimento do principal diferencial desse *chatbot* para os outros: a especialização nos documentos de acessibilidade digital. O banco de dados *Pinecone*, de onde viriam as informações, é povoado a partir de um processo chamado de *embedding.* Esse processo se define pela transformação de informações textuais em vetores numéricos de tamanho fixo que o banco vetorial (e consequentemente a IA) compreendem a partir da busca semântica. Apesar de sua importância no projeto, esse código entraria em execução apenas uma vez durante o processo de desenvolvimento, pois o povoamento do banco vetorial seria feito com informações fixas e que, em tese, não necessitariam de manutenção posterior.

A geração dos vetores tinha duas etapas: a transformação desses dados simples em números e a exportação deles para o banco de dados vetorial. Além da construção desse novo algoritmo, foram necessárias alterações no código base de comunicação com a IA para o adequar ao novo contexto.

Para a conversão dos documentos, foram utilizados pacotes *Node.js* com essa função específica, simplificando e automatizando o processo em algumas poucas linhas de código.

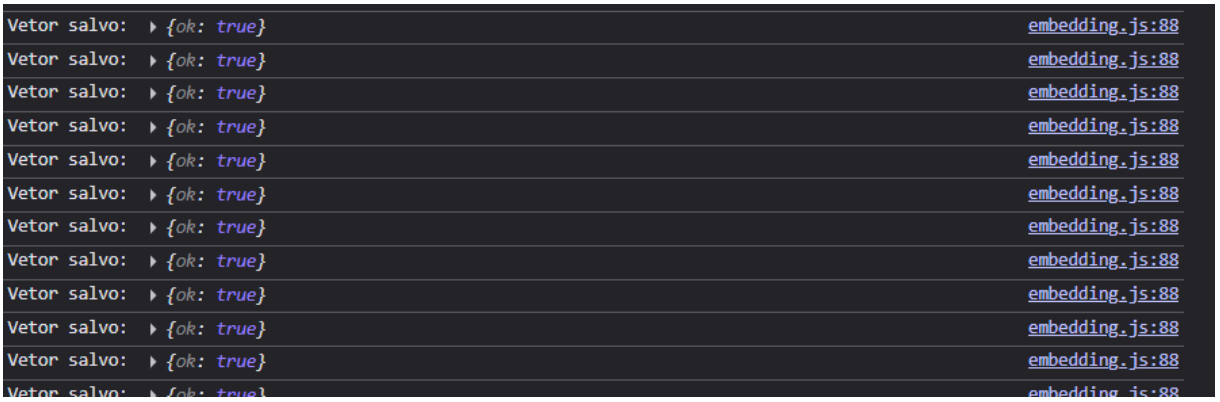
FIGURA 2 - Geração vetorial



Fonte: Desenvolvida pelo autor (2026).

Com os vetores sendo gerados com sucesso, agora era necessário enviá-los pela rede ao *Pinecone*. O processo utilizou o pacote SDK oficial *Node.js* do banco *Pinecone*. Este se mostrou um desafio relativamente maior por conta de sua complexidade comparado ao que havia sido utilizado até o momento com *Node.js*. No entanto, foi possível superar as dificuldades e realizar essa etapa de forma funcional e de acordo com o esperado.

FIGURA 3 - Envio dos vetores



Fonte: Desenvolvida pelo autor (2026).

Apesar de funcional, o método utilizado apresentou um grande problema: os vetores criados por meio do *Node.js* eram pequenos demais para o conteúdo dos PDFs que populam o banco de dados, o que causou erros de interpretação por parte da IA quando foram realizados testes de funcionamento no sistema. A situação foi imediatamente registrada como algo que necessitaria de mudanças no futuro, mas que não era de urgência por causar erros apenas em situações específicas.

4.3.3 Conclusão da primeira fase

Após mais alguns testes de funcionamento, foi observado que as metas estipuladas para esse momento do desenvolvimento foram alcançadas e agora era momento de partir para as próximas etapas. Entre os problemas notados, o maior destaque foi, inevitavelmente, a incompatibilidade do volume de informações dos arquivos PDF e do tamanho do recorte de dados, o que veio a ser resolvido mais tarde no desenvolvimento.

4.4 A SEGUNDA VERSÃO

O próximo passo foi desenvolver o *backend* do sistema, adicionando ao código um banco de dados que armazenasse os dados de conversa e perfil do usuário, além de uma metodologia de login segura e funcional, respeitando a lei geral de proteção de dados (LGPD) adotada no Brasil.

Embora os planos iniciais sugerissem a escrita do código utilizando o PHP nativo, a estratégia foi reformulada após uma sugestão do supervisor de estágio para, no lugar da linguagem nativa, utilizar o *framework* Laravel. A decisão foi tomada com base no ganho de tempo proporcionado pelas ferramentas prontas, além de sua simplicidade quando comparado ao PHP original, transformando tarefas que tomariam dias de trabalho em poucas linhas de código.

O aprendizado do *framework* aconteceu durante o próprio período de estágio, utilizando a carga horária para os treinamentos, que foram realizados com base em cursos disponíveis *online* e projetos práticos.

4.4.1 Sistema de *login*

Ainda em conversa com o supervisor de estágio Lael sobre a abordagem do *backend*, foi sugerida a implementação do sistema de *login* utilizando os serviços de API da *Google*, sob os argumentos de que, além de ser uma alternativa mais simples para os próprios usuários, não precisando criar novas contas e senhas, utilizar uma tecnologia externa e confiável aumentaria a segurança do sistema, visto que nenhum dado sensível seria armazenado no banco de dados do próprio

sistema. Por consequência, o uso da API também agilizaria o processo de desenvolvimento, pelo fato de todas as ferramentas necessárias já estarem prontas para uso, havendo apenas a tarefa de codificar o comportamento do sistema quando estabelecidas certas condições. Apesar da existência de um sistema de *login*, era fundamental que houvesse a possibilidade de utilizar o sistema sem estar conectado à uma conta, por ser uma das exigências iniciais do projeto.

Para o desenvolvimento foi necessário o uso do *Google Cloud*, plataforma essa que oferece a API para *login* denominada *OAuth 2.0* e as configurações necessárias para seu uso. Também foi utilizado o pacote *socialite* do *composer*, cuja função é oferecer ferramentas de controle do fluxo *OAuth* diretamente pelo código. No ambiente do *Google Cloud*, foi necessário criar um projeto específico para a aplicação, configurar e gerar as credenciais de acesso (*Client ID e Client Secret*). Além disso, foram mapeadas as URLs de redirecionamento autorizadas, garantindo que os tokens de autenticação transitassem apenas entre domínios confiáveis durante os testes locais e na versão de produção.

Na execução, a funcionalidade já é vista na página inicial do sistema. Ao abrir o projeto, o usuário é direcionado para uma página com as opções “*login* com *Google*” e “seguir como visitante”, cujo diferencial seria o armazenamento ou não das conversas e mensagens do usuário, além de pequenas mudanças de comportamento da IA baseado em interações passadas. Também foram feitas seções condicionais e exceções para cenários em que o usuário cancela a autenticação no meio do processo ou ocorrem falhas de comunicação com a API. Nesses casos, o sistema redireciona o usuário com mensagens de erro, mantendo o funcionamento da aplicação

.

FIGURA 4 - Tela de *login*



Fonte: Desenvolvida pelo autor (2026).

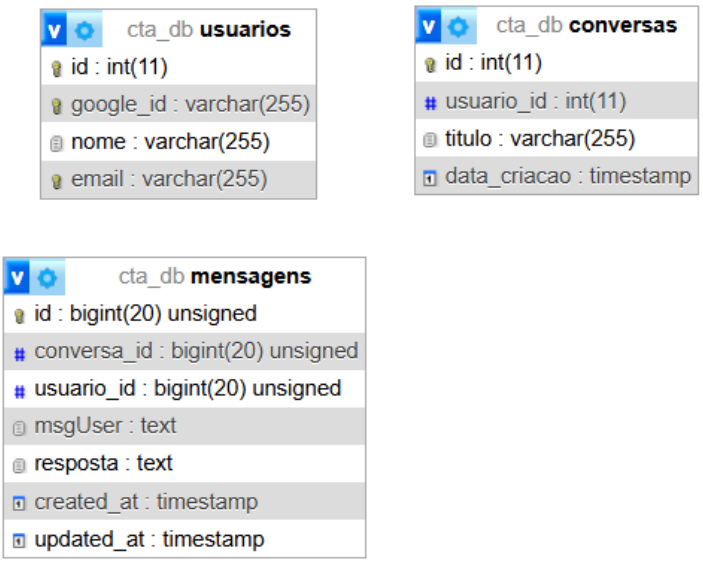
4.4.2 Banco de dados

O armazenamento de informações foi feito inicialmente utilizando como base o MySQL, linguagem de banco de dados relacional. Primeiramente, foi necessária a modelagem básica, planejando as ligações entre tabelas de usuários e suas subsequentes, como a de conversas. Como a *Google* já possui uma estrutura básica de tabela de usuário recomendada para quando o *OAuth* 2.0 está em uso, ela foi o ponto de partida inicial.

O resultado teve três tabelas essenciais, que englobava todos os dados que seriam necessários armazenar para o desenvolvimento e para a experiência sem erros do usuário: Usuários, em que se armazena apenas dados não sensíveis como o nome, *e mail*, ID da *google*; Conversas, onde se encontram o nome da conversa, data de atualização, além dos valores que se ligam às mensagens; E, por fim, mensagens, onde cada registro inclui a resposta do *bot* e o *prompt* juntos, automaticamente. Todas essas ligadas entre si por valores chaves denominados *foreign keys* exclusivos de cada registro.

A arquitetura final apresentou estabilidade para os dados, porém a velocidade de busca e necessidade de uma estrutura de dados fixa poderia vir a ser um problema futuro.

FIGURA 5 - Esquematização do banco de dados



Fonte: Desenvolvida pelo autor (2026).

4.4.3 Barra lateral

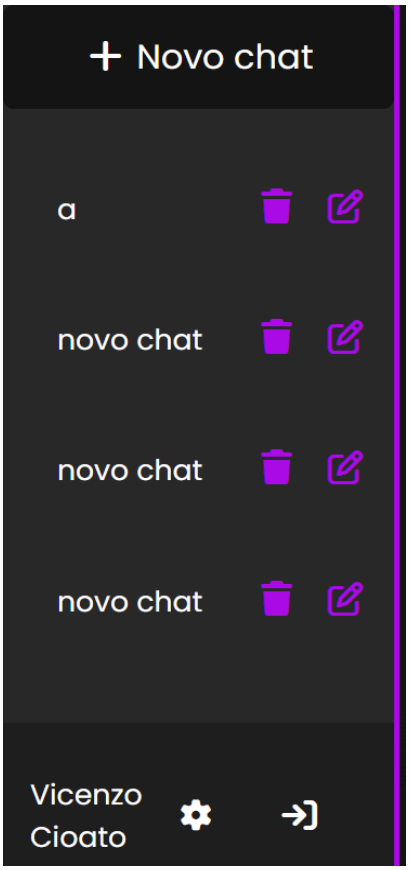
Sendo um elemento essencial de navegação na interface entre chatbots, o desenvolvimento da *sidebar* foi indispensável para uma versão que se propunha a ser utilizável por usuários. Ela foi pensada com base nos outros *chatbots*, analisando pontos negativos e positivos de cada um para chegar à um resultado simples e que cumprisse sua função de forma amigável ao usuário. O desenvolvimento foi inicialmente feito utilizando o *Livewire*, biblioteca do *Composer* que permite a criação de componentes de interface diretamente com *Laravel*, adicionando novos comandos para tornar isso possível.

O elemento finalizado apresentava, além da navegação entre conversas, a opção de gerenciamento sob as mesmas, permitindo ao usuário que ele excluísse e editasse o nome das conversas a qualquer momento, com poucos cliques. Também foi incluído, na região inferior, informações sobre a conta e um botão para efetuar o *logout*, ferramenta essa que é essencial para a navegação do usuário no sistema.

Apesar de funcional, a forma como o *Livewire* funciona causaria problemas em escala maior em produção, por conta da dependência do servidor para renderizar um elemento de interface. Quando combinada com a alta taxa de usuários, isso acabaria causando problemas de desempenho do sistema em um

geral. O problema só foi detectado posteriormente, e exigiu uma reconstrução quase total do elemento.

FIGURA 6 - Barra lateral



Fonte: Desenvolvida pelo autor (2026)

4.4.4 Conversas e mensagens

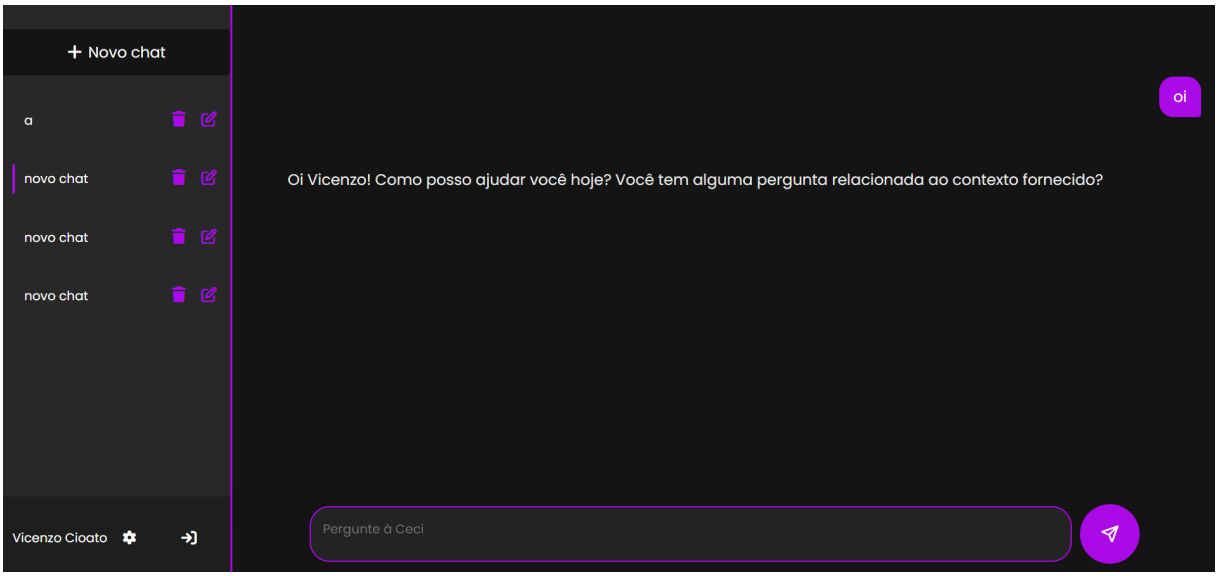
Como o banco de dados já havia sido corretamente preparado, os registros de mensagens já possuíam as devidas ligações entre elas e suas respectivas conversas, o que tornou o desenvolvimento do sistema de conversas e mensagens um processo simples e que trouxe poucos problemas, sendo mantido quase sem alterações até o estado final do sistema.

4.4.5 Protótipo da interface

Pelo estado relativamente avançado do desenvolvimento quando comparado ao previsto, o momento foi considerado apropriado para uma pausa no desenvolvimento de *features* por um breve tempo para focar em coisas menores, como a interface do sistema e refatorações pontuais no código.

Quanto à interface, apesar de não ter tido a paleta de cores deixada de lado, ela não foi foco desta etapa, ao invés disso se priorizou a estruturação de como os elementos deveriam se comportar e encaixar na tela. Houve a inspiração nos outros *chatbots* famosos anteriormente mencionados, visto que a estruturação da página de *chat* já se tornou convencional em todos, mas pela disparidade em quantidade de mecânicas, a readequação dos elementos fez com que as mudanças naturalmente viessem. Além disso, se priorizou uma interface mais simplista e com resposta durante as interações, com elementos como interatividade maior dos elementos com o *mouse*, contraste maior entre texto e fundo e visibilidade maior dos botões.

FIGURA 7 - Visão geral da interface



Fonte: Desenvolvida pelo autor (2026)

Além da interface, o algoritmo de comunicação com a IA também foi completamente refatorado, tendo o seu código migrado de um servidor *Node.js* para um método presente na classe *controller* da mensagem PHP. Essa mudança, além de trazer melhorias para a organização do código, diminuiu a exigência de *hardware* necessário para a compilação do código ao concentrar todo o processo em um único servidor, também aumentando a velocidade de desempenho geral de execução das chamadas.

4.5 TERCEIRA VERSÃO

Com a base do projeto bem construída e estruturada, os próximos passos foram de aprimoramento sobre tudo o que já havia sido feito, com maior parte das mudanças sendo focadas na melhor experiência do usuário e na migração de recursos externos ao código, como o modelo de IA e o banco de dados utilizado.

4.5.1 Uso do MongoDB

Após uma avaliação do progresso do sistema, o colega de trabalho Jason Piloti sugeriu a substituição do *MySQL* pelo banco de dados não-relacional MongoDB, por conta do seu tempo de resposta mais rápido em situações de buscas volumosas e pela não-exigência de uma estrutura fixa de dados na inserção de um registro, o que o torna ideal para programas onde o usuário pode ter registros vazios, como é nos *chats*. Ademais, os bancos no MongoDB possuem mais camadas de segurança mais complexas do que o *login* exigido pelo *MySQL*, dificultando ataques externos.

O MongoDB oferece uma alta flexibilidade para que programadores o utilizem nos mais diversos ambientes de desenvolvimento, sanando todas as possíveis necessidades. Inicialmente, a estratégia adotada foi a instalação nativa do serviço completo de banco de dados diretamente na máquina. No entanto, esta causou conflitos internos durante o próprio processo de instalação do pacote, que impediam completamente o seu uso. Diante dessa situação, então, optou-se por migrar para a

conteinerização via *Docker*, mudança essa que eliminou as incompatibilidades e simplificou o controle do banco de dados.

Poucos ajustes no código foram necessários na migração de banco de dados, visto que o *eloquent* do *Laravel* uniformiza os comandos para gerenciamento de banco de dados. A estrutura das *models* permaneceram praticamente idênticas às originais. Essa padronização reduziu drasticamente o tempo de refatoração do sistema. Assim, a migração do MySQL para o MongoDB ocorreu de forma simples, permitindo o foco nas próximas etapas.

4.5.2 Modelo de IA local

Conforme previsto no planejamento inicial do projeto, seria necessária a adoção de um modelo de IA local para que a manutenção inteiramente local, um dos principais objetivos do projeto, fosse atingido com sucesso. Para poder implementar esta, a tecnologia utilizada foi o *Ollama*, uma ferramenta *open-source* que disponibiliza o *download* e a execução de diversos modelos de IA diretamente pela máquina local, desde seus modelos próprios e leves até versões conhecidas e de alto desempenho, como o *Gemma, Deepseek* e *qwen*.

Para o desenvolvimento a LLM escolhida foi o *llama-3*. A decisão foi fundamentada com base em seus baixos requisitos mínimos de utilização, alta velocidade para a geração e por ser o mesmo utilizado na API da *groq*, o que diminuiria a quantidade de mudanças necessárias no código. Embora a máquina disponibilizada pelo CTA contasse com 32 *gigabytes* de memória RAM, ainda se tratavam de diversos processos internos e externos interligados com alta exigência de processamento. Enquanto ainda em produção, o *llama-3* se mostrou apropriado gerando respostas rápidas e de acordo com a base de informações, sendo inegavelmente a melhor opção do momento.

A conexão do *Ollama* ao código foi feita por meio do pacote do *composer ollama-Laravel,* o qual é desenvolvido e mantido pela comunidade e permite que o sistema Laravel interaja com modelos de IA facilmente, em poucas linhas de código.

4.5.3 Refatoração de código

Como mencionado em capítulos anteriores, o *Livewire* funciona construindo a estrutura HTML inteiramente pelo lado do servidor, o que causava aumento da latência e um *delay* notável até mesmo no servidor local. Por conta disso, foi necessária uma reformulação em todo o fluxo da *Sidebar*, utilizando apenas a interação padrão entre o *Laravel* e o *Javascript*. A troca ocorreu sem problemas e dentro do tempo previsto, conseguindo de forma bem-sucedida tornar o *lag* para as operações basicamente imperceptíveis.

4.5.4 Recursos de usabilidade e interface

Com base nos principais *chatbots* do mercado, foram adicionadas ao sistema mecânicas de auxílio ao usuário com o objetivo de tornar o sistema mais amigável ao cliente final. Após análises, concluiu-se que as adições necessárias seriam a edição de mensagem, cópia rápida de *prompts* e respostas para a área de transferência, transcrição do texto das respostas em áudio e modais de confirmação para operações de alteração e remoção de dados. Todos estes, com exceção da edição de *prompts*, por se tratar de uma complexa operação de dados, foram feitos com bibliotecas e funções nativas do *Javascript* e HTML, de forma que sua inclusão afetou apenas o lado do cliente sem modificar o desempenho do *site* diretamente.

Também foram realizadas alterações na interface, mantendo a base da estrutura mas com ajustes pontuais e substituindo a paleta desconexa pelo esquema de cores tradicional dos *sites* oficiais do instituto federal.

FIGURA 8 - Tela de *login* definitiva

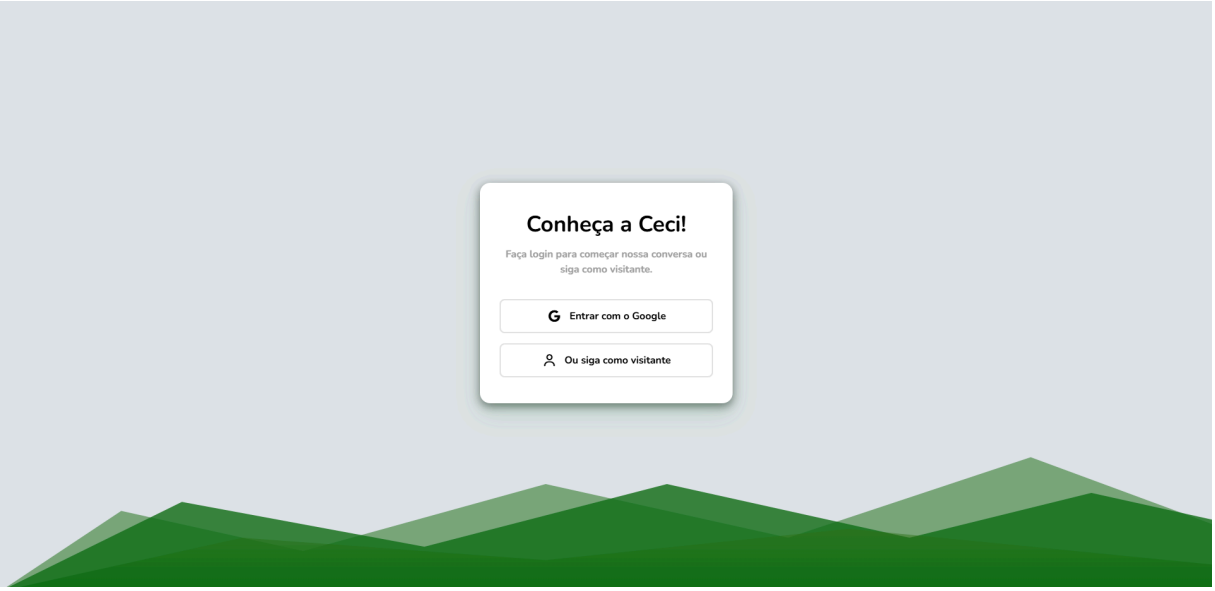
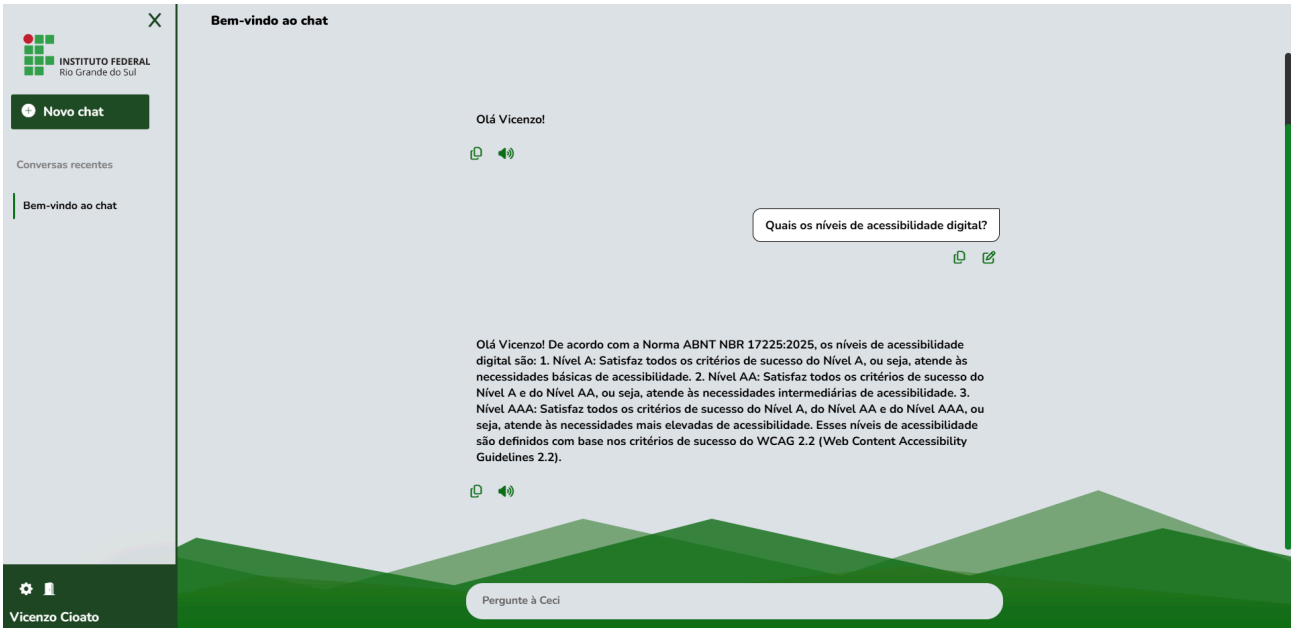


FIGURA 9 - Aba de conversa



4.5.5 Documentação com agente de IA

Ao final do período de estágio, foi recomendado pelo colega Jason, por fins de aprendizado, que a documentação do projeto fosse feita por um agente de IA. Para essa última tarefa foi utilizado o *Copilot*, o qual é integrado diretamente ao *VScode* e possui uma generosa taxa de uso gratuito, incluindo as funções de criação automática de documentos e leitura dos arquivos diretamente pelo projeto, sem a necessidade de enviar manualmente os arquivos para que o processo acontecesse.

Além da documentação, o agente foi utilizado para a realização de testes com o projeto. A partir dessa análise, foi possível encontrar pontos fracos que passaram sem ser notados anteriormente, como a falta de condicionais de verificação em determinados trechos do código, o que se tornaria grave em uma eventual hipótese em que este sistema entra em produção sem alteração alguma nesses métodos.

5 CONCLUSÃO

Ao fim do período de estágio, o sistema foi entregue em um estado considerado acima do esperado, seja em funcionalidades ou qualidade, porém ainda assim longe de poder ser considerado pronto.

5.1 DIFICULDADES DO PROCESSO

O processo de desenvolvimento se mostrou desafiador durante todo o período, por ter se baseado inteiramente em conhecimentos que foram adquiridos conforme o sistema demandou, necessitando de muito estudo e planejamento. Encontrar soluções, meios e ferramentas gratuitas que suprissem a necessidade do momento ao mesmo tempo em que ofereciam um resultado de qualidade, ao mesmo tempo em que se elaborava todo o fluxo de um tipo de sistema completamente diferente do habitual previamente conhecido foi com certeza um processo desafiador, mas edificante.

Fora da esfera da codificação, também houve problemas relacionados ao *hardware*. Em dado momento fora necessária a mudança de computador para que fosse suportado o uso da IA local no lugar de uma API externa. Ademais, a rede *wi-fi* do instituto federal bloqueou requisições básicas das quais o projeto era fortemente dependente.

5.2 PERSPECTIVA DO PROJETO

O sistema atualmente se encontra em um repositório privado do *Github* e conta com as suas principais mecânicas inteiramente desenvolvidas e funcionais, podendo até ser considerado pronto para uso, apesar de ainda serem necessárias

melhorias. Já é possível criar contas ou seguir como visitante, interagir com o *bot* sem problemas e conseguir respostas que vão perfeitamente de acordo com os documentos que a alimentam, além de uma personalização básica de perfil.

Todavia, ainda há diversas melhorias a serem feitas para que seu propósito de informar facilmente sobre a acessibilidade digital seja inteiramente cumprido. Além das anteriormente mencionadas brechas de segurança presentes no sistema, há uma lista de mudanças necessárias no projeto. Elas vão desde polimento do código e reapropriação de arquivos, adição de camadas extras de UX para o usuário, funcionalidades ausentes até reapropriação da interface para o *design system* oficial do CTA. Cabe ressaltar que a interface do projeto não teve suas condições de acessibilidade avaliadas, apesar de ser o tema central da ferramenta. Essa lacuna é uma das prioridades para o andamento futuro do projeto e que não foi solucionada durante o próprio período de estágio pela falta de especialização no assunto.

Portanto, apesar de bem encaminhado, esse sistema ainda necessita de diversas adições, alterações e melhorias para que possa ser considerado realmente finalizado.

5.3 CONSIDERAÇÕES FINAIS

Durante os quatro meses de estágio, foram aplicados não apenas os conhecimentos adquiridos ao longo do curso técnico em Informática para Internet, mas também novas habilidades desenvolvidas conforme a necessidade de explorar outros âmbitos da programação que certamente serão de grande valor na vida profissional. Toda a trajetória no CTA representou um importante marco de aprendizado e adaptação às expectativas do mercado de trabalho.

A criação de um software do zero, passando por todas as etapas de desenvolvimento, desde a idealização aos ajustes finais, permitiu compreender na prática como esse processo ocorre. Aprender os fundamentos da Inteligência Artificial e suas funcionalidades, além de colaborar em um projeto com potencial de gerar um impacto social positivo, foi sem dúvida uma experiência de grande valor.

Desta forma, pode-se considerar que o estágio cumpriu seus objetivos, proporcionando uma vivência profissional em um contexto real. Além da programação, serão levados aprendizados sobre a vida profissional como um todo.

REFERÊNCIAS

AWS. **Front-end x back-end — Diferença entre desenvolvimento de aplicativos — AWS**. Disponível em: <https://aws.amazon.com/pt/compare/the-difference-between-frontend-and-backend/>. Acesso em: 14 set.. 2026.

CHATGPT. **Conheça o ChatGPT**. Disponível em: <https://openai.com/pt-BR/index/chatgpt/>. Acesso em: 14 set.. 2026.

COODESH. **O que é biblioteca?**. Disponível em: https://coodesh.com/blog/dicionario/o-que-e-biblioteca/. Acesso em: 18 set. 2026

COMPOSER. **Introduction - Composer**. Disponível em: <https://getcomposer.org/doc/00-intro.md>. Acesso em: 18 set.. 2026.

CLOUDFLARE. **O que são embeddings no aprendizado de máquina?** Disponível em: <https://www.cloudflare.com/pt-br/learning/ai/what-are-embeddings/>. Acesso em: 14 set.. 2026.

GOOGLE. **O que é inteligência artificial (IA)?** Disponível em: <https://cloud.google.com/learn/what-is-artificial-intelligence?hl=pt-br>. Acesso em: 14 set.. 2026.

GITHUB. **GitHub - cloudstudio/ollama-Laravel: Ollama-Laravel is a Laravel package providing seamless integration with the Ollama API.** Disponível em: <https://github.com/cloudstudio/ollama-Laravel>. Acesso em: 14 set.. 2026.

GOVBR. **Acessibilidade Digital**. Disponível em: <https://www.gov.br/governodigital/pt-br/acessibilidade-e-usuario/acessibilidade-digital>. Acesso em: 14 set.. 2026.

GOVBR. **Guia de Boas Práticas para Acessibilidade Digital**. Disponível em: <https://www.gov.br/governodigital/pt-br/acessibilidade-e-usuario/acessibilidade-digital/guiaboaspraaticasparaacessibilidadedigital.pdf>. Acesso em: 14 set.. 2026.

GOV.BR. **Modelo de Acessibilidade**. Disponível em: https://www.gov.br/governodigital/pt-br/acessibilidade-e-usuario/acessibilidade-digital/modelo-de-acessibilidade. Acesso em: 18 set. 2026

GRANTMESTRENGTH. **Sudo para Windows**. Disponível em: <https://learn.microsoft.com/pt-br/windows/advanced-settings/sudo/>. Acesso em: 18 set.. 2026..

INACIO, Aylton. **O que é WCAG?** Acessibilidade para conteúdo Web. Disponível em: https://ayltoninacio.com.br/blog/o-que-e-wcag-acessibilidade-na-web. Acesso em: 18 set. 2026.

LARAVEL. **Installation | Laravel 13.x - The clean stack for Artisans and agents**. Disponível em: <https://laravel.com/framework/docs>. Acesso em: 18 set.. 2026.

LIVEWIRE. **Quickstart | Laravel**. Disponível em: <https://livewire.laravel.com/docs/4.x/quickstart>. Acesso em: 18 set.. 2026.

MONGODB. **Bem-vindo à documentação do MongoDB.** Disponível em: https://www.mongodb.com/docs/. Acesso em: 18 set. 2026.

NODE. **Node.js.js v20.2.0 Documentation**. Disponível em: <https://Node.jsjs.org/docs/latest/api/>. Acesso em: 14 set.. 2026.

OLLAMA. **llama3**. Disponível em: <https://ollama.com/library/llama3>. Acesso em: 14 set.. 2026.

PEDIGO, Mark. **MySQL vs MongoDB: Escolhendo o banco de dados certo para seu projeto**. Disponível em: <https://www.datacamp.com/pt/blog/mysql-vs-mongodb>. Acesso em: 14 set.. 2026.

PINECONE. **Pinecone Documentation - Pinecone Docs**. Disponível em: <https://docs.Pinecone.io/guides/get-started/overview>. Acesso em: 14 set.. 2026.

V-TRMYL. **Visual Studio Code**. Disponível em: <https://learn.microsoft.com/pt-br/shows/visual-studio-code/>. Acesso em: 14 set.. 2026.